blog (проект)

Стек: FastAPI, SQLAlchemy

Я подумал, что было бы неплохо создать небольшой блог на FastAPI. В качестве СУБД буду использовать PostgreSQL.
Для начала я создал структуру проекта:

```
.
├── __init__.py
├── app
│   ├── __init__.py
│   ├── config.py
│   ├── database
│   │   ├── __init__.py
│   │   └── models.py
│   ├── homepage
│   │   ├── __init__.py
│   │   └── homepage.py
│   └── post
│       ├── __init__.py
│       └── post.py
└── main.py
```

Ее я подсмотрел у чувака, который начал делать свой Pinterest: https://github.com/shutsuensha/shutsuensha.ru/tree/main. У него огромный проект - я пока не собираюсь делать именно такой. Мне будет достаточно, если удастся реализовать создание постов + какие-нибудь комментарии, авторизацию и лайки.

глава 1 посты

Я решил начать с постов. Для них создана отдельная папка - `post`. В ней будут храниться сами функции постов + схемы на Pydantic. Прежде чем писать функцию, которая будет отдавать все посты или добавлять новые посты, нужно сначала сделать структуру БД.

структура БД

Структура БД у меня будет храниться в папке `database` в `models.py`. Сейчас модели выглядят так:

```
from sqlalchemy import BigInteger, String, Text, CheckConstraint
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column


class Base(DeclarativeBase):
    pass


class Post(Base):
    __tablename__ = "post"
    post_id: Mapped[int] = mapped_column(BigInteger,
primary_key=True)
    title: Mapped[str] = mapped_column(String(150))
    content: Mapped[str] = mapped_column(Text())

    __table_args__ = (
        CheckConstraint("length(title) > 0",
name="chk_length_title"),
        CheckConstraint("length(content) > 0",
name="chk_length_content")
    )
```

Для таблицы `Post` пока что сделаны три простых поля - `post_id, title, content`:

1. `post_id` - `bigint` + `primary_key`. Такие настройки говорят ORM, что нужно прописать автоматическую генерацию ключей по полю + поставить в качестве типа поля `bigint`.
2. `title` - `String(150)` определяет, что в колонке в качестве типа должен стоять `varchar(150)`.
3. `content` - `Text()` определяет, что в колонке в качестве типа должен стоять `text`.
   В константе `__table_args__` прописаны проверки для двух колонок -

`title` и `content`. Проверки проверят, что во входящих значениях по этим полям больше нуля символов.

а как создать БД?

Чтобы создать БД в SQLAlchemy нужно сначала создать движок. Грубо говоря - это ссылка на БД в формате строки. Она выглядит примерно как-то так:

```
from sqlalchemy import create_engine
engine =
create_engine("postgresql+psycopg2://scott:tiger@localhost:5432/myda
tabase")
```

Компоненты ссылки:

1. `postgresql` - наименование СУБД.
2. `psycopg2` - движок, который будет использовать SQLAlchemy для подключения к БД.
3. `scott` - имя пользователя.
4. `tiger` - пароль для пользователя.
5. `localhost` - IP адрес хоста.
6. `5432` - порт.
7. `mydatabase` - название БД.
   Это все можно было бы закинуть в один файл с моделями, однако мне не очень хотелось превращать проект в помойку, поэтому я сделал отдельный файлик для запуска движка + запуска процесса создания таблиц.

```
.
├── __init__.py
├── app
│   ├── __init__.py
│   ├── config.py
│   ├── database
│   │   ├── __init__.py
│   │   ├── database.py  # новый файл здесь
│   │   └── models.py
```

```
│   ├── homepage
│   │   ├── __init__.py
│   │   └── homepage.py
│   └── post
│       ├── __init__.py
│       └── post.py
└── main.py
```

Туда я сразу закинул создание движка и создание сессий:

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker


engine = create_engine()
session = sessionmaker(bind=engine, expire_on_commit=False)
```

Что такое `session`?

сессии в SQLAlchemy

Сессия - это такая фича ORM, которая позволяет отправлять запросы в БД не по одному, а сразу скопом, отдавая на откуп ORM составление порядка и группировку запросов. Это означает, что ORM пойдет разбираться со всеми запросами, которые необходимо сделать к БД только когда мы обратимся к объекту сессии с просьбой закоммитить изменения в БД.
Сессия:

1. уберет лишние тысячу обращений в БД
2. отсортирует и сгруппирует запросы в нужном порядке
3. примет изменения в БД
   Такой подход называется `Unit of Work`.
   Еще одна позитивная сторона сессий - `Identity Map`. Карта идентичности гарантирует, что сессия будет наблюдать за данными, которые мы запрашиваем в код приложения из БД, и не позволит одним и тем же данным дублироваться в коде, забивая память.

гениальный settings.py

Наверное, было заметно, что в последнем экземпляре кода не заполнены скобки у `create_engine()`. Я оставил их пустыми намеренно, потому что увидел реальную техномагию, которую очень хочу применять теперь всегда. Когда я писал проекты раньше, то довольно часто делал файл `settings.py` или `constants.py`, где размещал все нужные приложению настройки и константы. Мне всегда казалось, что такой подход немого грубоват, но другого я не знал + всегда были большие проблемы, с которыми нужно было разобраться.
Однако я нашел парня, который сделал реальную бомбу в своем проекте - файл `config.py`: https://github.com/shutsuensha/shutsuensha.ru/blob/main/app/config.py. Кратко - он делает класс `Settings`, который унаследован от класса `BaseSettings` из Pydantic. Туда он заливает все основные константы по проекту. Но самое крутое, что все "секретные" константы берутся напрямую из файла `.env` с помощью интерфейса Pydantic.
Так как мне нужно так же хранить где-то свои секреты, я решил повторить успех и сделал такую же структуру в своем проекте:

```
from dotenv import load_dotenv
from pydantic-settings import BaseSettings, SettingsConfigDict


load_dotenv()

class Settings(BaseSettings):

    POSTGRES_DB_HOST: str
    POSTGRES_DB_NAME: str
    POSTGRES_DB_USER: str
    POSTGRES_DB_PASS: str
    POSTGRES_DB_PORT: int

    @property
    def POSTGRES_URL(self):
        return f"postgresql+psycopg://{self.POSTGRES_DB_USER}:
{self.POSTGRES_DB_PASS}@{self.POSTGRES_DB_HOST}:
```

```
{self.POSTGRES_DB_PORT}/{self.POSTGRES_DB_NAME}"

    model_config = SettingsConfigDict(env_file=".env",
env_file_encoding="utf-8")

settings: Settings = Settings()
```

Здесь есть сразу несколько интересных моментов:

1. библиотека `dotenv` устанавливается вместе с пакетом `pydantic-settings`.
2. пакет `pydantic-settings` - это независимое DLC для Pydantic, которое можно использовать, чтобы писать красивые настройки проекта.
3. `dotenv` и `pydantic-settings` взаимоинтегрированы поэтому внутри класса `Settings` можно использовать класс `SettingsConfigDict`, чтобы подтянуть настройки из существующего `.env` файла. Эта механика прописана в доках: https://docs.pydantic.dev/latest/concepts/pydantic_settings/#dotenv-env-support.
4. использование свойства (`@property`) для того, чтобы вернуть сформированную строку через интерфейс точечной нотации из класса.

Теперь я могу создать тестовую базу через `psql`:

```
postgres=# create user blog_admin with password 'mycoolp@ssword';
CREATE ROLE
postgres=# create database blog
postgres-# with
postgres-# encoding="UTF8"
postgres-# LC_COLLATE="ru_RU.UTF8"
postgres-# LC_CTYPE="ru_RU.UTF8"
postgres-# template=template0
postgres-# owner blog_admin;
CREATE DATABASE
```

И нужно открыть доступ к пользователю `blog_admin` в `pg_hba.conf`:

```
host    all        blog_admin      0.0.0.0/0          md5
```

Теперь иду заполнять `.env`. Он должен лежать рядом с нашим файлом конфигурации `config.py` и лучше сразу его добавить в исключения `.gitignore`:

```
POSTGRES_DB_HOST="192.168.64.1"
POSTGRES_DB_NAME="blog"
POSTGRES_DB_USER="blog_admin"
POSTGRES_DB_PASS="mycoolp@ssword"
POSTGRES_DB_PORT=5432
```

create_engine и создать БД

Теперь файл `database.py` выглядит так:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from app.config import settings


engine = create_engine(settings.POSTGRES_URL)
session = sessionmaker(bind=engine, expire_on_commit=False)
```

Все это очень круто и красиво, но я все еще не нашел нормального метода, который запускал бы коннект с БД всякий раз, когда я активирую сервер. Для начала я добавил две функции - одна создает таблицы и базу данных (`create_db_and_tables()`) (хотя база данных у меня уже есть), а другая выдает сессии для выполнения запросов (`get_session()`) + добавил специальную аннотированную константу - `SessionDep`:

```
def create_db_and_tables():
    Base.metadata.create_all(engine)


def get_session():
    with session_sync() as session:
        yield session
```

```
SessionDep = Annotated[session_sync, Depends(get_session)]
```

`SessionDep` в будущем будет использоваться для обращений к БД из функций приложений.
`create_db_and_tables()` нужно еще откуда-то вызвать. Доки по FastAPI подсказывают, что настроить вызов такой функции можно специальным образом - при запуске сервера. Это я пропишу в `main.py`:

```
@app.on_event("startup")
def on_startup():
    database.create_db_and_tables()
```

Однако в доках такой метод называют устаревшим. Вместо него предлагается использовать `lifespan`. Грубо говоря он выполняет ту же функцию, что и `on_startup` - перед тем как приложение начнет получать реквесты будут сделаны некие действия. В моем случае это будет удаление БД и создание новой БД:

```
from contextlib import asynccontextmanager

from app.database import database


@asynccontextmanager
async def lifespan(app: FastAPI):
    database.delete_db_and_tables()
    database.create_db_and_tables()
    yield


app = FastAPI(lifespan=lifespan)
```

Теперь все готово. Можно запустить сервер:

```
uvicorn main:app
```

Когда сервер будет запущен, в базе данных должна создаться таблица `post`. Я захотел это проверить, поэтому подключился к БД через `psql` и вывел список отношений:

```
blog=# \d
                  List of relations
 Schema |       Name        |   Type   |   Owner
--------+-------------------+----------+------------
 public | post              | table    | blog_admin
 public | post_post_id_seq  | sequence | blog_admin
(2 rows)
```

Таблица создана. Ура-ура.

проблемы

Моя первая проблема - неверная формулировка пароля для БД. Изначально я поставил пароль `mycoolp@ssword`, но не подумал о том, что строка, в которую помещается этот пароль вообще-то итак содержит `@`, но в качестве служебного символа:

```
"postgresql+psycopg2://scott:tiger@localhost:5432/mydatabase"
```

И это вызвало коллизию. Скорее всего эту строку можно как-то экранировать, но я не хотел этим заниматься на первых этапах, поэтому просто сменил пароль на БД.
Вторая проблема - пакет `psycopg2`. Это драйвер, который мы используем в SQLAlchemy и который по всей видимости нужно устанавливать отдельно. Однако его установка сама по себе может стать проблемой. Поэтому я воспользовался советом в сети - установить пакет `psycopg2-binary` вместо `psycopg2` и все сработало.

а как создать асинхронную БД?

Этот вопрос я задал потому, что во всех гайдах, которые я посмотрел на данный момент гайдеры вообще не вдаются в различия между синхронной и асинхронной версиями и сразу лепят `async` везде, где его можно влепить.

Такую же привычку унаследовал и коннект к БД. Чтобы приконнектиться к БД можно использовать асинхронный движок `sqlalchemy` + можно настроить асинхронное создание таблиц и асинхронную выдачу сессий.
Теперь мой файл `database.py` выглядит так:

```
from sqlalchemy.ext.asyncio import create_async_engine,
async_sessionmaker

from app.config import settings
from .models import Base


engine = create_async_engine(settings.POSTGRES_URL)

async_session_maker = async_sessionmaker(bind=engine,
expire_on_commit=False)


async def get_session():
    """
    Функция выдает сессии.
    """
    async with new_session() as session:
        yield session


async def setup_db():
    """
    Функция удаляет БД и создает снова новую.
    """
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)
        await conn.run_sync(Base.metadata.create_all)


SessionDep = Annotated[AsyncSession, Depends(get_session)]
```

Где я теперь использую `setup_db`? Все в том же `lifespan`. Функцию `lifespan` я разместил в файле `main.py`:

```
from fastapi import FastAPI
from contextlib import asynccontextmanager

from app.database import database

@asynccontextmanager
async def lifespan(app: FastAPI):
    await database.setup_db()
    yield


app = FastAPI(lifespan=lifespan)
```

Здесь опять понадобился асинхронный менеджер контекста + я добавил `await` к `database.setup_db()`.
Обязательно нужно поменять движок, который использует SQLAlchemy для получения доступа к БД. Строка, где он у меня упоминается находится в файле `config.py` в одном из свойств класса `Settings`:

```
    @property
    def POSTGRES_URL(self):
        return f"postgresql+asyncpg://{self.POSTGRES_DB_USER}:
{self.POSTGRES_DB_PASS}@{self.POSTGRES_DB_HOST}:
{self.POSTGRES_DB_PORT}/{self.POSTGRES_DB_NAME}"
```

Я заменил `psycopg2` на `asyncpg`.

написать post-метод

Теперь вопрос: а как я могу отправить post-метод в эту таблицу, чтобы начать писать в нее данные? Изначально я создал модель `Post` в файлике `database/models.py`:

```
from sqlalchemy import BigInteger, String, Text, CheckConstraint
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column


class Base(DeclarativeBase):
```

```
    pass


class Post(Base):
    __tablename__ = "post"
    post_id: Mapped[int] = mapped_column(BigInteger,
primary_key=True)
    title: Mapped[str] = mapped_column(String(150))
    content: Mapped[str] = mapped_column(Text())

    __table_args__ = (
        CheckConstraint("length(title) > 0",
name="chk_length_title"),
        CheckConstraint("length(content) > 0",
name="chk_length_content")
    )
```

Эта крутая модель, как я говорил должна создать таблицу в моей БД с такими полями, которые я описал. Кстати, SQLAlchemy сама конвертирует типы питоновских данных в типы данных SQL, но делает она это в пределах разумного, то есть, если я пишу `Mapped[int]` то полю в таблице будет задан тип `Integer` - для всех остальных integer-ов в SQL мне нужно дать пояснение, поэтому я написал `BigInteger`.
Теперь мне нужно написать сам запрос, который будет заполнять эти поля:

```
from fastapi import APIRouter

from .schemas import PostCreate
from ..database import database, models


router = APIRouter()


@router.post("/p")
async def add_post(post: PostCreate, session: database.SessionDep):
    new_post = models.Post(
        title=post.title,
        content=post.content
    )
```

```
    session.add(new_post)
    await session.commit()
```

Здесь написана асинхронная функция `add_post`. На вход она ожидает две сущности:

1. `PostCreate` - схема.
2. `SessionDep` - штука для зависимостей.
   Внутри функции происходят простые вещи:
3. `new_post` - берет модель для БД `Post` и передает в нее данные, которые были получены от пользователя и трансформированы в словарь автоматически в переменной `post`.
4. `session.add(new_post)` обращаемся к сессии с просьбой добавить новые данные в БД.
5. `await.commit()` - применяем данные к БД.

что такое схема?

Схема - это сущность Pydantic, которую FastAPI использует для валидации и форматирования входящих данных.
Например, у нас есть задача принять от пользователя данные и каким-то образом их верифицирвать. Можно было бы придумать некий скрипт, написанный вручную, который с помощью строковых методов или чего-то такого проверяет пошагово каждый экземпляр данных. А можно поступить проще и написать класс Pydantic:

```
from pydantic import BaseModel


class PostCreate(BaseModel):
    title: str
    content: str
```

Готово. Теперь можно использовать этот класс в качестве аннотации к функции FastAPI, чтобы FastAPI автоматически верифицировал данные в теле входящего запроса + приводил бы их к нужным типам.

что такое зависимость?

Когда мы обсуждаем зависимость в контексте `SessionDep`, которую я передал в параметры функции, нужно сначала вспомнить какое значение имеет эта переменная. Ее я прописал в файле `database.py`:

```
SessionDep = Annotated[AsyncSession, Depends(get_session)]
```

В данном случае мы как бы запрашиваем объект типа `AsyncSession` от функции `get_session`. Значение буквально говорит: переменная зависит от получения объекта `AsyncSession` из функции `get_session`. Сама функция `get_session` написана так:

```
async def get_session():
    async with async_session_maker() as session:
        yield session
```

Когда я пишу запрос к функции с параметром `session: SessionDep`, SQLAlchemy `get_session` сразу создает и отдает открывает сессию, с которой можно работать внутри функции.

пишу post-запрос

Чтобы написать POST запрос я использую `curl`:

```
curl -X POST -H "Content-Type: application/json" -d '{"title": "Мой первый пост!", "content": "Ура-ура!"}' http://127.0.0.1:8000/p
```

Параметры:

1. `-X` - параметр, который позволяет указать тип запроса.
2. `-H` - описываем дополнительный заголовок запроса - тип отправляемых данных.
3. `-d` - сами данные в нужном типе (в моем случае JSON).
   В самом конце стоит ссылка на эндпоинт. После отправки запроса на сервер должен придти отчет:

```
INFO:     127.0.0.1:55136 - "POST /p HTTP/1.1" 200 OK
```

получаю посты

Чтобы получить посты нужно написать GET запрос. Я написал его в том же файле `post.py` и изначально он выглядел так:

```
@router.get("/p")
async def get_posts(session: database.SessionDep):
    result = await session.execute(text("""
        select post_id, title, content
        from post
    """))
    return result.scalars.all()
```

Однако такой запрос не вернет ожидаемых словарей с полями - он вернет список с идентификаторами записей. Я сначала удивился, потому что своровал этот способ у парня, который сказал, что именно так мы сможем достать записи из БД. На самом деле так мы достаем только их номера.
Как же реально достать записи из БД? Для этого нужно опять воспользоваться схемой Pydantic. Я написал вторую схему в файле `schemas.py`:

```
from pydantic import BaseModel


class PostCreateSchema(BaseModel):
    title: str
    content: str

# вторая схема
class PostSchema(PostCreate):
    post_id: int
```

Так как у нас в таблице всего три поля, а для создания поста пользователю не нужно указывать `post_id` (потому что он указывается СУБД автоматически), я решил, что можно унаследовать схему поста от схемы создания поста и добавить всего одно поле - `post_id`.

Теперь нужно импортировать схему в файл с функцией и упомянуть ее в декораторе:

```
@router.get("/p", response_model=list[PostSchema])
async def get_posts(session: SessionDep):
    result = await session.execute(text("""
        select post_id, title, content
        from post
    """))
    return result.all()
```

Теперь FastAPI поймет к каким данным нужно преобразовать полученные из SQLAlchemy объекты (мы получаем итератор в результате работы `execute()`) и вернет красивый словарь:

```
[{"title":"Мой первый пост!","content":"Ура-ура!","post_id":1}]
```

ORM и raw запросы

Я послушал штук 200 видео про FastAPI и все гайдеры наперебой избегают использования сырых SQL запросов в коде. У этого может быть несколько причин:

1. использование ORM запроса благоприятно влияет на какие-то фишки внутри SQLAlchemy.
2. гайдеры не хотят напрягать новичков лишним синтаксисом.
3. использование ORM - стандарт для современного разработчика. Аргументом в пользу ORM часто выступает фраза "так проще, потому что мы пишем SQL запросы на Python", но т.к. я знаю SQL лучше, чем я знаю синтаксис SQLAlchemy ORM мне проще писать сырой SQL запрос и проще понять его в коде.
   Поэтому в части получения постов я написал именно сырой SQL запрос:

```
@router.get("/p", response_model=list[PostSchema])
async def get_posts(session: database.SessionDep):
    result = await session.execute(text("""
        select post_id, title, content
```

```
        from post
    """))  # вот тут мой запрос.
    return result.all()
```

А как написать такой же запрос на ORM? Здесь есть свои сложности.
Чтобы написать такой же запрос нужно обратиться к модели, которую я написал ранее в `models.py` и импортировать из SQLAlchemy конструкцию `select`, которая является калькой на `select` из SQL:

```
from sqlalchemy import select


@app.router("/p", response_model=list[PostSchema])
async def get_posts(session: database.SessionDep):
    query = select(models.Post)
    result = session.execute(query)
    return result.all()
```

Если написать запрос так, то он вернет ошибку валидации:

```
fastapi.exceptions.ResponseValidationError: 3 validation errors:
  {'type': 'missing', 'loc': ('response', 0, 'title'), 'msg': 'Field
required', 'input': (<app.database.models.Post object at
0x78f46526fb10>,)}
  {'type': 'missing', 'loc': ('response', 0, 'content'), 'msg':
'Field required', 'input': (<app.database.models.Post object at
0x78f46526fb10>,)}
  {'type': 'missing', 'loc': ('response', 0, 'post_id'), 'msg':
'Field required', 'input': (<app.database.models.Post object at
0x78f46526fb10>,)}
```

Из ошибки видно, что Pydantic модель, которая должна проверять данные из БД, не получает сами значения из полей БД. Вместо них она получает какой-то объект класса `Post` - `(<app.database.models.Post object at 0x78f46526fb10>,)`. То есть она буквально получает необработанный объект модели, который нужно по идее сначала распаковать и найти в нем нужные данные.
Чтобы это сделать я добавлю функцию `scalars()`:

```
return result.scalars().all()
```

Смысл ее работы в названии: скаляр - это одномерный массив. В контексте нашей функции к такому массиву будут преобразованы полученные из БД значения. Результат будет такой же, какой был при использовании SQL запроса. Так же, `scalars()` можно применить вместо `execute()`, если мы ожидаем получить именно простые значения, а не объекты модели:

```
result = session.scalars(query)
return result.all()
```

чтобы получить один пост

Получение одного поста я планирую сделать по индексу - запрос должен содержать параметр пути, в котором должен быть передан индекс. То есть запрос будет выглядеть примерно так:

```
http://127.0.0.1:8000/p/{index}
```

Здесь возникает несколько нюансов:

1. я хочу, чтобы в мою функцию попали именно числа, а не любой ввод.
2. данные придется достать из БД по индексу.
   Чтобы решить первую проблему нужно просто добавить проверку типов и конвертер пути к функции. Конвертер будет выглядеть так:

```
@router.get("/p/{index:int}")
```

Запись `:int` указывает, что этот шаблон пути подойдет только в том случае, если эту часть пути в пользовательском запросе можно преобразовать к числу. Дополнительно можно добавить типизацию внутрь функции:

```
@router.get("/p/{index:int}")
async def get_post(index: int, session: database.SessionDep):
```

Конвертер и типизация будут проверять входящий параметр на соответствие числовому типу.
Как написать запрос на получение поста с конкретным идентификатором? В SQL такой запрос выглядел бы так:

```
select post_id, title, content from post
where post_id = index;
```

Но я хочу написать такой же, но на ORM SQLAlchemy. Синтаксис ORM предлагает примерно схожую концепцию:

```
@router.get("/p/{index:int}")
async def get_post(index: int, session: database.SessionDep):
    query = select(models.Post).where(models.Post.post_id == index)
    result = await session.scalar(query)
    return result
```

глава 2 разнообразие БД

Я пока создал только одну таблицу в БД, но сразу понятно, что так сервисы с постами не работают. К каждому посту должен быть прикреплен его автор. Это означает, что помимо таблицы с постами должна быть реализована так же и таблица с пользователями.
Она будет выглядеть проще, чем модель постов, но это только пока:

```
class User(Base):
    __tablename__ = "users"

    user_id: Mapped[int] = mapped_column(BigInteger,
primary_key=True)
    username: Mapped[str] = mapped_column(String(200), unique=True)

    __table_args__ = (
        CheckConstraint("length(username) > 0",
name="chk_length_username"),
    )
```

Стоит дать пару комментариев по поводу именования таблиц. Таблицу `post` я назвал именно `post` изначально, а для таблицы `users` я выдал название во множественном числе. Почему? Потому что таблицу `user` PostgreSQL не даст создать в БД. Теперь, после того как я назвал во множественном числе таблицу пользователей, я должен назвать во множественном числе и таблицу постов, потому что название таблиц должны быть согласованными по синтаксису - так будет проще обращаться к таблицам, когда их станет очень много.
Далее, я поставил после `CheckConstraint` запятую - это способ создать кортеж в Python. `__table_args__` требует на вход именно последовательность, поэтому если не указывать запятую после `CheckConstraint` SQLAlchemy просигналит об ошибке. Так происходит, потому что Python автоматически не учитывает скобки, если в них упомянут только один элемент.

как связать БД?

Теперь возникает вопрос: а как указать, что поле из таблицы постов теперь будет ссылаться на пользователя из таблицы пользователей? В SQL такая механика реализовывается через конструкцию `references` при создании БД. В этом случае таблица постов создавалась бы так:

```
create table posts(
    post_id bigint generated always as identity primary key,
    title varchar(200) not null check(length(title) > 0),
    content text not null check(length(content) > 0),
    user_id bigint not null references users(user_id)
)
```

Однако как реализовать то же самое, но на языке моделей SQLAlchemy? Я написал так:

```
class Post(Base):
    __tablename__ = "posts"

    post_id: Mapped[int] = mapped_column(BigInteger,
primary_key=True)
    title: Mapped[str] = mapped_column(String(150))
    content: Mapped[str] = mapped_column(Text())
```

```
    user_id: Mapped[int] = mapped_column(
            BigInteger,
            ForeignKey("users.user_id", ondelete="cascade")
    )

    __table_args__ = (
        CheckConstraint("length(title) > 0",
name="chk_length_title"),
        CheckConstraint("length(content) > 0",
name="chk_length_content")
    )
```

Объяснение по `user_id`:

1. `ForeignKey` - название отражает термин "внешний ключ".
2. Первым аргументом я передал поле первичного ключа из таблицы `users`.
3. Вторым аргументом я передал настройку `ondelete="cascade"`. Она означает, что при удалении пользователя так же будут удалены все связанные с ним данные, т.е. в данном случае его посты.
4. Тип данных ключа `BigInteger` должен совпадать с типом данных в таблице `users`.

обновление модели Post

Для базы данных по постам я дописал еще поле - поле для фиксации времени создания поста:

```
class Post(Base):
    __tablename__ = "posts"

    post_id: Mapped[int] = mapped_column(BigInteger,
primary_key=True)
    title: Mapped[str] = mapped_column(String(150))
    content: Mapped[str] = mapped_column(Text())
    created_at: Mapped[datetime] = mapped_column(
        TIMESTAMP(timezone=True),
        default=lambda: datetime.now(timezone.utc)
    )  # здесь
```

```
    user_id: Mapped[int] = mapped_column(
            BigInteger,
            ForeignKey("users.user_id", ondelete="cascade")
    )

    __table_args__ = (
        CheckConstraint("length(title) > 0",
name="chk_length_title"),
        CheckConstraint("length(content) > 0",
name="chk_length_content")
    )
```

Объяснение по `created_at`:

1. `TIMESTAMP` - сущность из SQLAlchemy, которая определяет тип поля.
2. `default` - параметр, который будет устанавливать для поста значение по умолчанию, если такое значение не было передано вместе с запросом.
3. Внутри `default` лежит анонимная `lambda`, которая будет брать время из функции `now()` модуля `datetime`.
   Остается только одна проблема - часовой пояс. И я так и не нашел как можно вставить нормальную поправку на время, потому что все мои попытки дополнить конструкцию `datetime.now()` нормальной таймзоной в итоге превратились все в то же дефолтное время UTC. Стоит прочитать заметку: https://stackoverflow.com/a/462028/30499559
   После установки времени я пошел дополнять схемы для `post`:

```
from datetime import datetime

from pydantic import BaseModel


class PostSchema(BaseModel):
    post_id: int
    user_id: int  # поле для юзера
    created_at: datetime  # поле для времени
    title: str
    content: str

    class Config:
```

```
        from_attributes = True


class PostCreateSchema(BaseModel):
    user_id: int  # поле для юзера
    title: str
    content: str
```

И немного исправил функцию, которая должна добавлять посты в БД:

```
@router.post("/p")
async def add_post(post: PostCreateSchema, session:
database.SessionDep):
    new_post = models.Post(
        title=post.title,
        content=post.content,
        user_id=post.user_id  # новое поле
    )
    session.add(new_post)
    await session.commit()
```

приложение user

Для приложения `user` я сделал отдельную папку. Структура всего проекта теперь выглядит так:

```
.
├── app
│   ├── config.py
│   ├── database
│   │   ├── database.py
│   │   ├── __init__.py
│   │   ├── models.py
│   ├── homepage
│   │   ├── homepage.py
│   │   ├── __init__.py
│   ├── __init__.py
│   ├── post
```

```
│   │   ├── __init__.py
│   │   ├── post.py
│   │   └── schemas.py
│   └── user  # новое приложение
│       ├── __init__.py
│       ├── schemas.py
│       └── user.py
├── __init__.py
├── main.py
└── requirements.txt
```

Внутри - те же файлы, которые я делал для `post`. В `schemas.py` сохранены схемы для создания пользователя и выдачи информации по пользователю:

```python
from pydantic import BaseModel


class UserSchema(BaseModel):
    user_id: int
    username: str

    class Config:
       from_attributes = True


class UserCreateSchema(BaseModel):
    username: str
```

Кстати, `from_attributes` - это настройка, которая обеспечивает интеграцию схем Pydantic и моделей ORM. Она необходима, потому что Pydantic изначально не может производить валидацию экземпляров класса, т.е. он не может взять экземпляр и проверить его атрибуты. Так происходит в стандартном варианте, потому что Pydantic ждет на вход либо словарь, либо JSON объект.
Далее, я сделал функцию для создания пользователя в `user.py`:

```
@router.post("/user")
async def add_user(user: UserCreateSchema, session:
database.SessionDep):
    new_user = models.User(
        username = user.username
    )
    session.add(new_user)
    await session.commit()
```

миграции

Я должен был об этом задуматься раньше, но мне так нравилось писать простое приложение с простыми SQL запросами, с использованием `psql`, что я совсем забыл обо всем остальном.
Когда приложение масштабируется и в него требуется вносить правки, зачастую эти самые правки касаются состояния БД. Например, внезапно на ресурс пришла куча пользователей и СУБД начала захлебываться от количества запросов. Причины могут быть разные. Одна из них - плохая инфраструктура БД.
Например, у нас есть одна масштабная таблица для информации по всем пользователям. В этой таблице находятся сущности, которые мы постоянно запрашиваем из БД, например, имя пользователя и его почта, а еще сущности, которые нам практически не нужны при работе пользователя с сервисом, например, его место рождения, дата свадьбы и т.д. Такую таблицу стоит перестроить (разбить на таблицы поменьше и соединить их связью один к одному) и неплохо было бы протестировать новую постройку.
В данный момент мои таблицы удаляются и снова создаются при запуске сервера. Такой подход хорош для быстрой практики, однако на реальном сервисе - это отстой. Неплохо было бы иметь такой инструмент, который может проанализировать наши модели, составить по этим моделям запросы к БД, сделать эти запросы, а еще сохранить предыдущее состояние БД, чтобы к нему можно было откатиться если что.
Именно такую функцию выполняет встроенный в Django инструмент для миграций. В FastAPI такого встроенного инструмента нет + у меня есть внешний ORM от SQLAlchemy, который было бы неплохо еще и интегрировать с новым инструментом. К счастью мудрые белые люди сделали Alembic.

что такое Alembic?

Alembic - это универсальный инструмент для выполнения миграций. В идеале он должен посмотреть в наши модели, проанализировать изменения, которые мы внесли в них, затем создать миграционные файлы (буквально запросы SQL) и применить их на нашу БД. Формально - это именно то, что сейчас выполняет моя функция на бесконечное удаление и создание таблиц в БД, но гораздо лучше, потому что выполнение миграций - это контролируемый процесс, который внесет обновления в БД, только после соответствующей команды.

что такое миграция?

Миграция - это буквально файл, который содержит информацию об изменениях, которые нужно внести в БД на основании анализа моделей SQLAlchemy. Его будет генерировать Alembic по шаблону, который уже есть по умолчанию при установке Alembic + который мы можем изменить при большом желании (он называется `script.py.mako`).